

PORTFOLIO 2026 · AI ALGORITHM & ENGINEERING

把复杂业务，拆成可运行的算法系统

吴宇歌 · YUGE WU

大模型算法 / 多智能体工程 / 数据与知识系统 · 新加坡国立大学 科学人工智能硕士

01 **问题定义**
模糊诉求 → 可解问题

02 **模型训练**
训练 · 对齐 · 压缩

03 **系统工程**
把模型装进能跑的平台

04 **评测治理**
结果可量化 · 数据可检索

05 **FDE 交付**
原型 → 客户生产环境

06 **行业落地**
在金融场景跑通闭环

04 **EchoMind 运行时**
意图路由 · 分层记忆 · 工具层

05 **问对 • 访谈辅导**
议题树 · 流式 ASR · 报告

01 **异常行为识别**
YOLOv8 · SlowFast · INT8

02 **大模型全链路**
LoRA · DPO · KV Cache

03 **彩翼 • 智能体平台**
五层架构 · L1–L4 路由

07 **政务数据治理**
76.3 万公告 · 六阶段

08 **NUS 文本编码**
κ 0.86 · 47,950 条

06 **知栈 • 知识库**
pgvector · SSE · 成本权衡

09 **AI 量化交易**
因子 · 回测 · 风控

10 **金融风控模型**
评分卡 · GBDT · PSI

能力传承：彩翼架构 → EchoMind | FDE 交付 → 知栈 | 多模态压缩 → 微调蒸馏 | 数据治理 → 风控特征

200%
评审监管业务效率提升

98.5%
合规审查精确率

76.3万
条公告完成结构化治理

15.5%
单请求成本下降

五类 · 十个项目 · 2026.11

github.com/Sheroisshero

目录

五类十个项目，每个项目按「背景 — 介绍 — 亮点 — 原理与架构 — 可视化 — 可公开技术细节」展开。

01 / 算法类

01 **AI 远程异地评审专家异常行为识别** 实战交付

02 **大模型全链路工程手册** 课程实战复现

02 / 开发类

03 **彩翼 · 公共采购智能体协同管理平台** 实战交付

04 **EchoMind · 多 Agent 客服运行时** 自研

03 / 产品类

05 **问对 · 访谈实时辅导** 开源二次开发

06 **知栈 · 企业知识库问答** 课程实战复现

04 / 数据类

07 **政务采购数据治理** 实战交付

08 **47,950 条文本的 LLM 辅助编码** 实战交付

05 / 行业类

09 **AI 量化交易系统** 课程实战复现

10 **金融风控算法模型** 课程实战复现

实战交付

在职期间的真实项目，有真实用户与上线记录，涉敏内容已脱敏。

自研

独立完成设计与实现的系统，可现场演示并讲解实现细节。

课程实战复现

基于公开课程或开源项目的实现与改造，标注基线来源，不冒充原创。

算法类

2 个项目

01 · AI 远程异地评审专家异常行为识别

02 · 大模型全链路工程手册

开发类

2 个项目

03 · 彩翼

04 · EchoMind

产品类

2 个项目

05 · 问对

06 · 知栈

数据类

2 个项目

07 · 政务采购数据治理

08 · 47,950 条文本的 LLM 辅助编码

行业类

2 个项目

09 · AI 量化交易系统

10 · 金融风控算法模型

PORTFOLIO 2026 • AI ALGORITHM & ENGINEERING

把复杂业务，拆成可运行的算法系统



吴宇歌

YUGE WU · 2026

大模型算法 / 多智能体工程

懂业务、能落地：从模型训练一路做到行业交付

学历	新加坡国立大学 · 科学人工智能硕士（AI for Science）
经历	7 年 · 政务 AI 4 年 · 智能体开发 2 年
实习	Google gTech Ads（FDE）· Shopee（搜索算法）
证书	软考高级系统架构师 · FRM

七年在做同一件事：把 AI 落到能验收的地方——四个政务场景从 0 到上线，也在 Google 与 Shopee 用算法解决真实的商业指标问题。

200% 评审监管业务效率提升	**98.5%** 合规审查精确率
76.3万 条公告完成治理	**15.5%** 单请求成本下降

VALUE CHAIN 一条链，十个项目

从业务问题到行业落地，每个节点都标出我在其中承担的角色、用到的技术栈和已经拿到的结果。

01 **问题定义** — 模糊诉求 → 可解问题

02 **模型训练** — 训练 · 对齐 · 压缩
- 01 异常行为识别 — YOLOv8 · SlowFast · INT8
- 02 大模型全链路 — LoRA · DPO · KV Cache

03 **系统工程** — 把模型装进能跑的平台
- 03 彩翼 • 智能体平台 — 五层架构 · L1–L4 路由

04 **评测治理** — 结果可量化 · 数据可检索
- 07 政务数据治理 — 76.3 万公告 · 六阶段
- 08 NUS 文本编码 — κ 0.86 · 47,950 条

05 **FDE 交付** — 原型 → 客户生产环境
- 06 知栈 • 知识库 — pgvector · SSE · 成本权衡

06 **行业落地** — 在金融场景跑通闭环
- 09 AI 量化交易 — 因子 · 回测 · 风控
- 10 金融风控模型 — 评分卡 · GBDT · PSI

能力传承：彩翼架构 → EchoMind | FDE 交付 → 知栈 | 多模态压缩 → 微调蒸馏 | 数据治理 → 风控特征

实战交付 · 自研 · 课程实战复现

2015 – 2019	**中南大学 • 信息管理与信息系统**	专业第一毕业，毕业论文被评为优秀
2022.10 – 2026.07	**广州市政府采购中心**	平台架构师 → 首席创新官；主导彩翼平台从 0 到 1
2026.04 – 至今	**新加坡国立大学 • AI for Science 硕士**	方向：深度学习算法与科学智能
2026.07 – 至今	**Google gTech Ads / Shopee**	前沿部署工程师实习 / 搜索与推荐算法实习

算法落地

多模态异常识别 Recall 85.1%、大模型全链路 165 章、边缘推理 15 FPS——模型不止跑在 notebook 里。

系统与产品

彩翼平台从 0 到 1、EchoMind 运行时、知栈知识库——把能力装进业务真正能用的系统。

FDE 交付

Google 3 个客户项目交付、单请求成本 -15.5%——在客户现场把原型推到生产环境。

接下来怎么读：后面十页，每个项目都按同一套结构展开——项目背景与业务意义 → **我的角色与核心贡献** → 亮点与业务影响 → 基础原理与架构 → 可视化 → 可公开的技术细节。带「查看代码 ↗」的项目可直接跳到 GitHub 仓库。

01 / 算法类

算法类

从多模态异常识别到大模型全链路：既做模型，也管数据、训练、推理与评测。

实战交付 01 · 算法类 · 广州市政府采购中心 · 项目负责人 / 组长 · 2025.03 – 2026.07 查看代码 ↗

AI 远程异地评审专家异常行为识别

把“看不见的评审现场”变成可预警、可追溯的结构化报告

我的角色与核心贡献 项目负责人 / 组长

- **定路线**：把「靠人盯屏」的监管诉求拆成视频 / 音频 / 日志三路可计算指标，确定融合判级规则与黄橙红三级预警
- **攻难点**：异常样本仅占 5%，用重采样 + Focal Loss + 分类别阈值把 Recall 拉到 85.1%，而不是统一设 0.5
- **推落地**：统筹算法、开发、交付三条线，30 台边缘设备 15 FPS 稳定运行，报告全自动生成，已在广州公共资源交易中心上线

项目背景

远程异地评标是政府采购的改革方向：专家分散在不同地点、通过音视频系统评审。监管的难点从“现场盯着”变成了“屏幕上找”——操作手机、交头接耳、擅自离席、不当言论这些行为，靠人工盯屏既看不全、也留不下证据。

项目介绍

一套视觉 + 音频 + 文本三路融合的异常行为识别系统，部署在评审现场边缘设备上，把异常行为从“发现”到“预警”再到“成文”全链路自动化，最终输出可追溯的《疑似异常行为分析报告》。

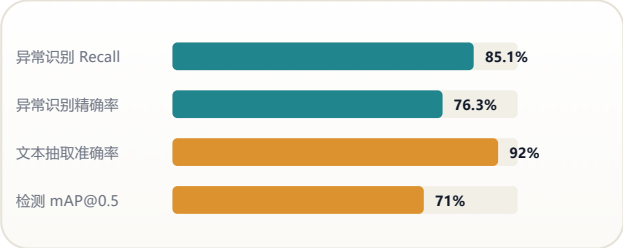
可公开的技术细节

- 不平衡学习：过采样 + Focal Loss，按类别调阈值而不是统一 0.5
- 时序行为窗口：16 帧滑窗 + 重叠采样，解决“动作跨秒”问题
- ASR 敏感词：同音字、变体写法、上下文否定一起判
- 模型压缩：INT8 量化 → 结构化剪枝 → 蒸馏三步，精度回补靠小学习率微调
- 数据与合规：3000 小时标注视频，涉敏画面脱敏后入库

项目亮点

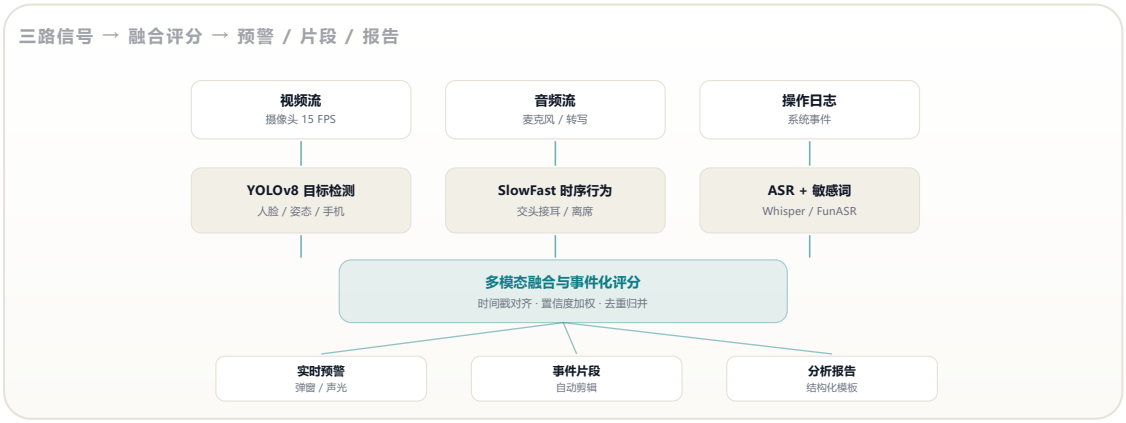
- **极端不平衡下可用**：异常样本仅占 5%，通过重采样与损失设计把异常识别 Recall 做到 85.1%、精确率 76.3%。
- **能上边缘设备**：YOLOv8 经 INT8 量化 + 剪枝 + 蒸馏从 6.2M 压到 2.1M，延迟降 40%，30 台设备 15 FPS 稳定运行。
- **不止于报警**：采集→识别→预警→剪辑→报告全自动，异常片段自动截取并绑定事件，业务效率提升 200%。
- **成果沉淀**：实用新型专利 2 项、发明专利 1 项，已在广州公共资源交易中心落地。

关键指标



基础原理与架构

视觉路用 YOLOv8 做人脸 / 姿态 / 手机检测，SlowFast 做时序行为识别；音频路用 Whisper/FunASR 实时转写并匹配违禁词；文本路做语义与规则双判。三路结果按时间戳对齐、置信度加权后融合成事件，再触发预警、切片和报告生成。



课程实战复现　02 · 算法类 · 个人项目 · 165 章技术体系 + 实验复现 · 2024 – 2026

大模型全链路工程手册

从数据配比到 KV Cache：把训练、微调、对齐、推理、评测串成一条能复现的链路

我的角色与核心贡献　165 章技术体系 + 实验复现

- **搭体系**：自己把散落的论文与课程整理成 5 系列 165 章，每个环节都落到「参数怎么选、实验怎么设计」
- **做复现**：QLoRA 微调 ChatGLM3-6B、PEFT、量化、DeepSpeed、MoE 逐项跑通，记录显存 / 吞吐 / 精度变化
- **给判断**：把选型边界写成可查的结论——什么时候 LoRA 够用、什么时候必须全参、DPO 与 RLHF 差在哪

项目背景

大模型落地最常断在两个地方：会用 API 但不懂训练，看得懂论文但跑不动实验。数据怎么配、LoRA 怎么设、DPO 为什么比 RLHF 省、KV Cache 到底吃多少显存——这些问题不打通，工程上就只能猜。

项目介绍

把大模型的全链路拆成 5 个系列、165 章：推理优化（量化 / KV Cache / 投机采样）、训练数据工程、LoRA/QLoRA 微调、DPO 偏好优化、SFT 数据配比与课程学习。每章都配参数选择与实验设计，并以课程实验（QLoRA 微调 ChatGLM3-6B、PEFT、量化、DeepSpeed、MoE）做复现验证。

可公开的技术细节

- LoRA / QLoRA：rank / alpha / target_modules 的取舍；4bit NF4 + 双重量化 + paged optimizer，单卡跑 6B
- SFT 数据配比：按难度分层做课程学习，先易后难 + 回放防遗忘
- DPO：β 与参考模型的关系，偏好对构造比超参更影响结果
- KV Cache / 投机采样：显存 ≈ 2 × layers × heads × head_dim × seq_len × batch × dtype；接受率决定加速比
- 评测：golden set + LLM-as-Judge + 人工抽检三层，避免单一指标失真

项目亮点

- **成体系**：5 个系列 165 章，覆盖预训练数据 → SFT → 对齐 → 推理 → 评测，不是碎片笔记。
- **可复现**：QLoRA 微调、Transformers 量化、DeepSpeed、MoE 逐项跑通，记录显存、吞吐、精度变化。
- **带判断**：同一问题给出选型依据——什么时候用 LoRA、什么时候必须全参、DPO 与 RLHF 的取舍边界。

基础原理与架构

链路是：数据工程 → SFT（LoRA/QLoRA）→ 偏好对齐（DPO/RLHF）→ 推理优化（量化 / KV Cache / 投机采样）→ 评测闭环。每个环节的输出都是下一环节的输入，评测结果反哺数据配比，形成闭环。



开发类

平台架构与运行时：一个面向真实行业的智能体底座，一个可运营的多 Agent 系统。

实战交付 03 · 开发类 · 广州市政府采购中心 · 平台架构师 / 首席创新官 · 2022.10 – 2026.07

查看代码 ↗

彩翼 · 公共采购智能体协同管理平台

全国首个公共采购智能体平台：从 0 到 1 搭起“智能体 + 数据 + 算力”的协同底座

我的角色与核心贡献　平台架构师 / 首席创新官

- **定架构**：设计五层架构（数据基座 → 模型基座 → 平台 → 应用管理 → 生态），把准入、算力、数据放进同一底座
- **抓合规**：用 RAG 混合检索替代规则引擎，合规审查精确率 28% → 98.5%，并建 2000 条标注集做回归
- **推上线**：推动 13 个智能体协同上线，覆盖交易额 100 亿+，沉淀软著 6 项、专利 6 项

项目背景

公共采购年规模超 40 万亿，其中政府采购 3.37 万亿，但数字化采购渗透率只有 11.5%。行业里 AI 工具是碎片化的：文件审查一个系统、质疑回复一个系统、比价又一个系统，数据不通、责任不清，合规场景下没人敢用。

项目介绍

彩翼是面向监管、采购人、代理机构、供应商四类主体的公共采购智能体协同平台，2025 年 4 月上线。规划 27 个智能体、已上线 10 款，采用五层架构，把智能体准入、数据治理、算力调度和业务场景放进同一个底座。

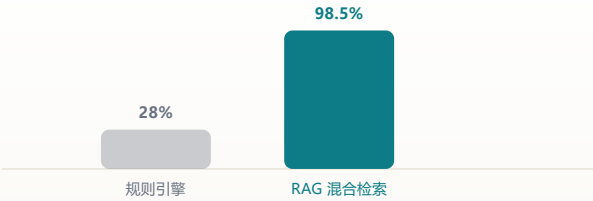
可公开的技术细节

- 智能体路由：按意图置信度分 L1–L4 四级，低置信度升级到更强模型而不是直接答
- RAG 混合检索：向量 + BM25 + 知识图谱 + 重排，按场景调权重
- Function Call：schema 版本化管理，失败重试 + 参数校验 + 兜底话术
- 评测：2000 条标注集 + 多模型 A/B，改动必须跑回归
- 第三方智能体准入：资质 → 技术 → 合规 → 场景实测 → 试运行五级
- 安全：XSS 五层防御、输出内容审核、操作全链路留痕

项目亮点

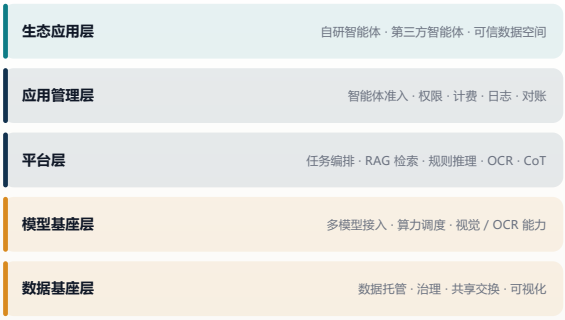
- **合规审查从“靠人”到“可量化”**：RAG 混合检索把合规审查精确率从 28% 提升到 98.5%，Top-5 召回率 89%。
- **多智能体真跑起来**：13 个智能体协同，L1–L4 路由，Function Call 成功率 94%，日均调用万次级。
- **数据底座**：4 个渠道 6 个库，治理 76.3 万条公告、53.6 万份合同，支撑模型训练与检索。
- **能算账**：单次推理成本压到 10 元以内，依托国产算力；平台覆盖交易额 100 亿+。
- **成果**：软著 6 项、专利 6 项（实用新型 2 / 发明 4），2026 数据要素大赛全国三等奖。

关键指标



基础原理与架构

五层架构自下而上：数据基座层负责托管、治理与共享交换；模型基座层做多模型接入与异构算力调度；平台层提供任务编排、RAG 检索、规则推理、OCR；应用管理层管准入、权限、计费、日志；生态应用层承载自研与第三方智能体。



自研 04 · 开发类 · 个人项目 · Python FastAPI + Java + React · 2026

EchoMind • 多 Agent 客服运行时

可观测、可评测、可降级：把“聊天壳”做成一套能运营的系统

我的角色与核心贡献　Python FastAPI + Java + React

- **从零写**：意图三路融合、Agent 路由、记忆分层、工具层，全栈 Python FastAPI + Java + React 自己实现
- **拆问题**：把「答非所问」拆成意图 / 路由 / 答案可用 / 拒答四类指标，分开定位、分开优化
- **做成能运营的**：加路由降权与 LLM-as-Judge 评测，系统出问题能定位、能降级、能回归

项目背景

大多数客服机器人是单轮问答壳：问价格能答，一问到跨业务的组合问题就答非所问。根因不是模型不行，而是意图、知识、工具、记忆、评测这五件事没有编排在一起，出了问题也没法定位。

项目介绍

EchoMind 是一个多 Agent 客服运行时：先做细粒度意图识别，再按意图路由到不同 Agent，按需触发 RAG，读写分层记忆，动态注入业务 Skills，并在线上持续监控、评测、必要时降权。

可公开的技术细节

- 三路意图：规则兜底、向量召回相似问、LLM 判语义，三路投票 + 置信度校准
- 路由降权：某 Agent 近期失败率升高时自动降权，把流量切给备用 Agent
- 记忆写入策略：只写“可复用的事实”，避免把寒暄灌进长期记忆
- Skills 热加载：业务规则不改代码，reload 即生效
- 工具层：TTL 缓存 + 熔断 + 结果重排，防止单工具拖垮整条链路
- 评测：意图 / 路由 / 答案可用 / 拒答得当四类指标分开看

项目亮点

- **意图先行**：规则 + 向量 + LLM 三路融合，输出 intent / intent_group / urgency / entities，而不是只给一个标签。
- **路由驱动**：General / Technical / Billing / Escalation 四类 Agent，按置信度与风险决定升级路径。
- **记忆分层**：Redis 存工作记忆，ChromaDB 存情景记忆与画像，避免长上下文硬塞。
- **可运营**：在线监控采集指标，异常时自动路由降权；LLM-as-Judge 做端到端评测。

关键指标



基础原理与架构

请求进入后先读记忆，再做意图识别；按意图决定是否检索知识库，然后路由到对应 Agent；Skills 按意图注入，工具层带缓存、熔断、重排；回复生成后写回记忆，同时进监控与评测。



产品类

从技术到可用：把访谈与知识库做成能交付给人使用的东西。

开源二次开发　05 · 产品类 · 个人产品化改造 · 基于开源 xiaozhi-fde-talk · 2026

问对 • 访谈实时辅导

访谈进行时，它替你想着下一个问题

我的角色与核心贡献　基于开源 xiaozhi-fde-talk

- **选基线**：认定 xiaozhi-fde-talk 的流式链路可用，把精力放在场景化改造而不是重写一遍
- **补核心**：设计议题树 + 覆盖度计算，让提示只问一个未覆盖的问题，而不是泛泛给建议
- **做交付**：串起 ASR → 辅导 → 报告 → 导出与 Docker 部署，让 FDE 能直接带进客户现场

项目背景

FDE、售前、咨询每天在做客户访谈，但访谈质量高度依赖个人经验：新手容易漏问关键约束，老手也常在被带偏后忘记回到提纲。录音转写工具只解决“记下来”，解决不了“问到位”。

项目介绍

问对是一个访谈实时辅导工具：开麦即转写，边听边分析对话结构，提示“下一个该问什么、哪些关键点还没问到”，访谈结束自动生成结构化需求报告，支持 Markdown / HTML / Word 导出。

可公开的技术细节

- 流式 ASR：分段送音 + 增量返回，字幕延迟控制在秒级
- 议题树：把访谈提纲拆成树，节点带“必问 / 可选 / 已覆盖”状态
- 提示生成：基于未覆盖节点 + 最近对话，约束模型只问一个问题
- 报告结构：背景 / 现状 / 痛点 / 约束 / 决策链 / 下一步
- 部署：Docker Compose + Nginx，ASR/LLM/OCR 三个配置档可切换
- 二次开发定位：在原开源项目基础上做场景化配置与交付流程改造，不冒充原创

项目亮点

- **实时提示而非事后复盘**：把访谈提纲变成议题树，边聊边判断覆盖度，缺口实时提醒。
- **数据不出内网**：FunASR 本地流式推理，适合带进客户现场和受监管行业。
- **聊完就有交付物**：自动生成结构化需求报告，减少事后整理与二次沟通。
- **一键部署**：Docker Compose 起全栈，注册即可用，可接豆包 / Qwen 等商用 ASR 与 LLM。

基础原理与架构

前端负责实时录音与字幕上屏，WebSocket 推流到后端；ASR 流式转写后进入辅导引擎，引擎按议题树计算覆盖度、生成提示；会话结束后汇总为结构化报告并导出。



课程实战复现 06 · 产品类 · 个人工程复现 · React 19 + NestJS + LangChain.js + pgvector · 2026

知栈 • 企业知识库问答

每个答案都能回到原文

我的角色与核心贡献 React 19 + NestJS + LangChain.js + pgvector

- **做选型**：对比专用向量库与 pgvector 同库，按规模与运维成本给结论，而不是照抄方案
- **搭全栈**：React 19 + NestJS + LangChain.js 四层链路，SSE 流式 + Zod 契约，前后端不靠口头约定
- **守标准**：以「答案能回到原文片段」为第一评测标准，chunk 元数据带 kbId / chunkNo 支持回溯

项目背景

企业知识散在 PDF、Excel、Wiki 里，搜索靠关键词，问一句“这个流程要走几步”就只能人工翻。专用向量库方案又要多维护一套存储，业务库和向量库双写，一致性出了问题很难查。

项目介绍

知栈是一个 TypeScript 全栈 RAG 知识库：文档入库、自动切片、向量化、混合检索、带引用回答，SSE 流式输出。存储只用一份 PostgreSQL + pgvector，业务元数据与向量表同库，复刻类火山方舟知识库的核心链路。

可公开的技术细节

- 切片：RecursiveCharacterTextSplitter，按标点与段落递归，保留 chunkNo 便于回溯
- 检索：按 kbId 过滤后再做相似度，避免跨库串答案
- 流式：SSE 推送 token，前端边收边渲染，首字延迟明显下降
- 契约层：Zod schema + 共享 DTO，前后端不靠口头约定
- FDE 视角：先问规模与运维现状，再决定要不要引入专用向量库
- 评测：以“答案可回溯到原文片段”为第一标准，而不是只看回答流畅度

项目亮点

- **同库存储**：knowledge_bases / documents / rag_chunks 一张 PG 里管完，避免双写不一致。
- **链路透明**：Loader → Splitter → Embedding → Retrieval → Prompt → Streaming，每步可观测、可替换。
- **多知识库隔离**：chunk 元数据带 kbId / documentId / chunkNo，检索按 kbId 过滤。
- **可交付**：pnpm monorepo，前后端共享 DTO 与流式事件类型，一条命令起 PG。

关键指标

存储选型：双库 vs 同库

专用向量库 + 业务库	PostgreSQL + pgvector
· 两套存储	· 一套存储
· 双写一致性	· 事务一致
· 运维成本高	· 中小规模够用

选型判断：知识库规模在千万级 chunk 以内、且已有 PG 运维能力时，同库存储的收益大于专用向量库的性能优势。

基础原理与架构

前端 React 19 做知识库、文档、切片、检索、问答五个页面；API 层 NestJS 提供 SSE 问答、CSV/XLSX 入库；RAG 引擎用 LangChain.js 串起 Loader → Splitter → Embedding → Retrieval → Prompt；存储层用 pgvector 表存向量与元数据。

四层架构：前端 / API / RAG 引擎 / PG + pgvector



同一份 PostgreSQL 里既有 knowledge_bases / documents，也有 rag_chunks 向量表——少一套双写，就少一类一致性事故。

数据类

先治理再建模：数据资产化，以及让 LLM 参与定性分析但保留人的判断。

实战交付 07 · 数据类 · 广州市政府采购中心 · 数据治理负责人 · 2023 – 2026

政务采购数据治理

先把数据变成能用的资产，再谈模型

我的角色与核心贡献　数据治理负责人

- **定方法**：把治理拆成盘点 → 标准 → 清洗 → 主数据 → 质量 → 安全六阶段，每阶段有可交付物
- **落标准**：数据元、代码项、编码规则、命名规范四类标准，统一跨系统口径
- **备弹药**：治理 76.3 万条公告、53.6 万份合同，沉淀 2000 条标注样本，让模型评测有基准

项目背景

采购数据散在 4 个渠道、6 个库：公告一套口径、合同一套口径、供应商信息又在另一个系统。字段命名不一致、重复记录多、关键字段缺失，直接拿去做 RAG 或训练，模型学到的就是噪声。

项目介绍

按“盘点 → 标准 → 清洗 → 主数据 → 质量 → 安全”六阶段推进治理，把 76.3 万条公告、53.6 万份合同整理成可检索、可训练、可追溯的数据资产，并沉淀 2000 条标注样本用于模型评测。

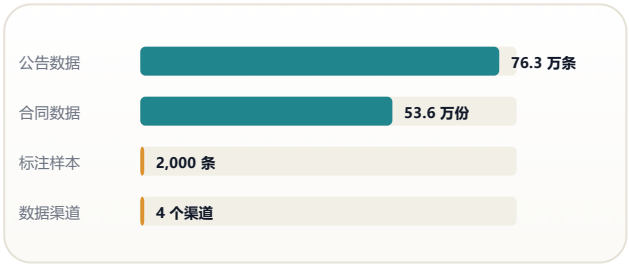
可公开的技术细节

- 盘点方法：业务调研 → 系统级 → 表级 → 字段级，四步落到字段清单
- 标准体系：数据元 / 代码项 / 编码规则 / 命名规范 / 信息资源目录
- 加工策略：全删全插、切片追加、拉链表三种，按更新频率选
- 质量规则：完整性、唯一性、一致性、及时性四类，配监控告警
- 安全：分级分类 + 脱敏 + 权限管控，敏感字段单独授权
- 元数据：血缘分析支撑影响评估，改表前先看下游

项目亮点

- **规模**：4 个渠道 6 个库，76.3 万条公告、53.6 万份合同完成结构化治理。
- **标准化**：建立数据元、代码项、编码规则、命名规范四类标准，跨系统口径统一。
- **可评测**：沉淀 2000 条标注样本，让“模型好不好”有基准而不是拍脑袋。
- **可追溯**：元数据 + 血缘，每个指标能追到源表和加工逻辑。

关键指标



基础原理与架构

六阶段流水线：先做业务调研与数据盘点（系统级 → 表级 → 字段级），再定标准，然后清洗与整合，抽取主数据（供应商、项目、商品），建立质量规则与监控，最后做分级分类与脱敏，形成可共享的数据资产。



实战交付 08 · 数据类 · 新加坡国立大学 · 研究助理 · Abel Yang 教授指导 · 2026.08 – 至今

47,950 条文本的 LLM 辅助编码

让大模型做初筛，让人只处理分歧

我的角色与核心贡献　研究助理 • Abel Yang 教授指导

- **改流程**：把传统开放式 → 轴心 → 选择性编码拆成可自动化步骤，人只处理分歧样本
- **定边界**：给模型正反例 + 强制引用原文，涉及价值判断的编码只做辅助、不做定论
- **控质量**：Cohen's kappa 0.86、主题覆盖 91%、人工复核压到 18%，在 47,950 条语料上验证

项目背景

社科定性研究依赖人工编码：读完几千条文本、逐条打标签、再做主题归纳。周期长、成本高，而且不同编码员之间的一致性很难保证。大模型能提速，但直接让模型打标签会引入幻觉和偏见。

项目介绍

把传统编码流程（开放式 → 轴心 → 选择性）拆解成可自动化的步骤：Embeddings 聚类先分堆，LLM 按编码本初筛，人工只复核分歧样本，再计算一致性、修订编码本，形成闭环。在 47,950 条文本上验证。

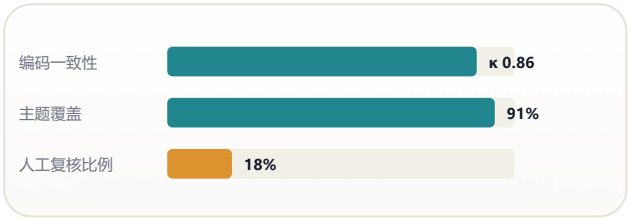
可公开的技术细节

- 编码本：先定义维度与取值，再让模型判断，避免“边标边改定义”
- Embedding 聚类：先用向量把语料分堆，降低逐条判断的成本
- 一致性：Cohen's kappa + 抽样复核，分歧样本回流修订
- 提示设计：给正反例 + 强制引用原文片段，减少无据编造
- 评测：一致性 / 覆盖率 / 人工复核比例三个维度一起看
- 边界：涉及价值判断的编码只做辅助，不做自动定论

项目亮点

- **规模**：47,950 条文本全量处理，人工复核比例压到 18%。
- **质量**：编码一致性 κ 0.86，主题覆盖率 91%。
- **方法**：明确人机边界——机器负责覆盖，人负责判断。
- **可迁移**：同一套流程可用于研报解析、舆情归因、客服会话分析。

关键指标



基础原理与架构

语料 → 清洗与分块 → Embedding 聚类 → LLM 按编码本初筛 → 人工抽样复核与分歧处理 → 一致性计算 → 编码本修订 → 主题归纳。评测用三维指标：一致性、覆盖度、复核成本。



行业类

在具体行业里跑通算法：量化交易的因子与回测，金融风控的可解释与监控。

课程实战复现 09 · 行业类 · 个人项目 · 12 周训练营复现 + 改造 · 2026

AI 量化交易系统

从因子到回测：把“看起来能赚”变成“能解释为什么赚”

我的角色与核心贡献　12 周训练营复现 + 改造

- **走全链路**：数据 → 因子 → 组合 → 回测 → 风控 → 模拟执行，每段都有产出物，而不是只写因子
- **防过拟合**：walk-forward 滚动窗口 + 样本外验证，用 IC / IR / 分层回测判断因子有效性
- **加另类信号**：用 LLM 抽公告与研报事件（类型 / 方向 / 置信度），人工抽检后进入信号

项目背景

量化门槛高在“链路”：数据、因子、组合、回测、风控、执行每一段都有坑。只做因子不回测，不知道是不是过拟合；只做回测不控回撤，实盘拿不住。

项目介绍

按 12 周训练营的路径完整走一遍量化系统：数据获取与清洗、因子构建、组合打分与约束、walk-forward 回测、回撤与仓位风控，并用 LLM 辅助研报与公告解析，做事件驱动的信号补充。

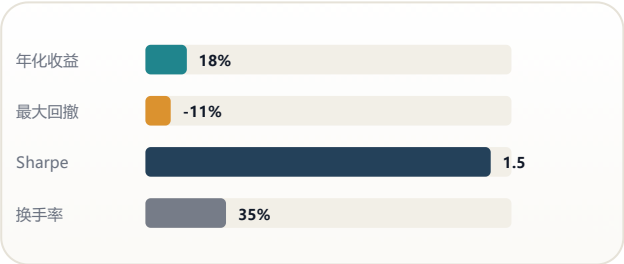
可公开的技术细节

- 因子处理：去极值 → 标准化 → 中性化，三步之后再进模型
- 有效性：IC / IR / 分层回测，三者一起看才不容易被骗
- 回测：walk-forward 滚动窗口，训练集与验证集严格分开
- 风控：最大回撤阈值、单票上限、行业偏离约束
- LLM 事件抽取：公告 → 事件类型 / 影响方向 / 置信度，人工抽检
- 复盘：每轮回测记录参数、样本区间、失效原因，避免重复踩坑

项目亮点

- **全链路**：数据 → 因子 → 组合 → 回测 → 风控 → 模拟执行，每段有产出物。
- **不过拟合**：walk-forward + 样本外验证，因子有效性看 IC / IR 而不只看收益。
- **能控风险**：最大回撤与仓位约束写进流程，而不是事后看。
- **LLM 加持**：用大模型解析研报与公告，抽取事件与预期差，作为另类信号。

关键指标



基础原理与架构

数据层拉行情与财务；因子层做动量、波动、质量三类因子并标准化；组合层按打分选股并加行业与个股权重约束；回测层用 walk-forward 滚动训练与验证；风控层设最大回撤、单票上限、换手上限；执行层做模拟盘验证。



课程实战复现 10 · 行业类 · 个人项目 · 评分卡 + 机器学习风控 + 部署 · 2026

金融风控算法模型

可解释、可监控、能上线：风控模型的三个硬约束

我的角色与核心贡献　评分卡 + 机器学习风控 + 部署

- **搭链路**：数据与标签 → 特征工程 → 模型 → 评估 → 监控 → 部署，六段完整复现
- **保可解释**：卡方分箱 + 单调性约束 + WOE，让每个拒绝理由能落到具体变量与分箱
- **建监控**：PSI 看漂移、KS 看衰减、坏账率看真实表现，三者一起判模型是否该迭代

项目背景

风控和推荐最大的区别是：推荐错了用户划走，风控错了要付出真金白银，还要能向监管解释。所以模型不只要能区分好坏，还要可解释、可监控、能部署。

项目介绍

按“数据与标签 → 特征工程 → 模型 → 评估 → 监控 → 部署”的链路完整复现金融风控建模：信用评分卡、消费金融风控、企业经营风险预测，以及模型部署的几种方式。

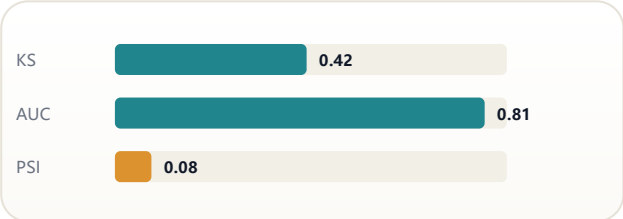
可公开的技术细节

- 分箱：卡方分箱 + 单调性约束，避免“分箱很好看但业务讲不通”
- WOE / IV：IV 筛变量，WOE 让逻辑回归能处理非线性
- 评估：KS、AUC、提升度、跨期稳定性，四项缺一不可
- 监控：PSI > 0.1 触发排查，KS 衰减超过阈值触发重训
- 拒绝推断：只有放款样本有表现，要用拒绝推断修正样本偏差
- 部署：批量（离线评分）、实时（API）、嵌入式（规则引擎内）

项目亮点

- **评分卡**：分箱 + WOE + 逻辑回归 + 分数映射，每个拒绝理由可解释。
- **机器学习风控**：GBDT 类模型提升区分度，用 SHAP 做可解释性补偿。
- **监控闭环**：PSI 看漂移、KS 看衰减、坏账率看实际表现。
- **部署**：批量评分、实时 API、嵌入式三种方式，按场景选。

关键指标



基础原理与架构

数据层整合申请、行为、征信三类数据并定义好坏标签；特征层做分箱、WOE 编码、IV 筛选；模型层用评分卡做基线、GBDT 做提升；评估层看 KS / AUC / 提升度；上线后持续监控 PSI 与 KS。



NOTE

关于项目性质

作品集里每个项目都标了类型，避免把“跟着课程做过”说成“从零自研”。

实战交付

在职期间的真实项目

有真实用户、真实数据、上线记录
能讲清需求来源、决策过程与结果
涉及未公开数据与内部系统的部分已做脱敏

自研

独立设计与实现的系统

代码、架构、评测均由本人完成
可现场演示与讲解实现细节
EchoMind 为完整自研项目

课程实战复现

在公开课程 / 开源基线上的实现

如实标注基线来源，不冒充原创
重点呈现工程理解、改造点与可迁移能力
面试时可展开讲“我改了什么、为什么”

CONTACT

吴宇歌 · Yuge Wu
新加坡国立大学 · 科学人工智能硕士（AI for Science）
方向：大模型算法 / 多智能体工程 / 数据与知识系统

github.com/Sheroisshero

说明

本作品集用于求职展示。涉及原单位的项目已做脱敏处理，不包含未公开的采购数据、客户信息与内部系统细节。

标注“课程实战复现”的项目基于公开课程或开源项目，个人贡献在于实现、改造与工程化验证。
版本：2026.11

以上，是过去七年里，十个可以拿出来聊的切片。